Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 03

Factorial Recursivo en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: April 29, 2026

Contents

1 Introducción **2**

2 Organización del programa **2**

3 Código fuente **2**

3.1 Programa principal (main.cpp) 2

3.2 Función ensambladora (factorial.asm) 3

4 Cómo funciona la recursión **4**

5 La Unidad de Punto Flotante (FPU) **5**

6 Ejemplo de ejecución **6**

7 Repositorio GitHub **7**

8 Conclusiones **7**

1 Introducción

Esta práctica consiste en calcular el **factorial** de un número usando dos lenguajes al mismo tiempo: **C++** (entrada y salida de datos) y **ensamblador x86** (operación matemática). La idea es ver cómo un programa de alto nivel puede delegar trabajo a código ensamblador, que está más cerca del hardware.

El factorial de un número es multiplicar ese número por todos los enteros positivos menores que él. Por ejemplo:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 \tag{1}$$

Matemáticamente, se puede definir de forma **recursiva** (es decir, usando la misma operación pero con un valor más pequeño cada vez):

$$x! = \begin{cases} 1 & \text{si } x \leq 1 \quad \text{(caso base)} \\ x \cdot (x-1)! & \text{si } x > 1 \quad \text{(paso recursivo)} \end{cases} \tag{2}$$

2 Organización del programa

El proyecto tiene dos archivos que trabajan juntos:

Table 1: Archivos del proyecto

Archivo	**Qué hace**
`main.cpp`	Le pide un número al usuario, llama a la función ensambladora y muestra el resultado.
`factorial.asm`	Contiene el cálculo del factorial escrito en ensamblador con instrucciones FPU.

El flujo del programa es sencillo:

1. El usuario escribe un número en la consola.
2. C++ recibe ese número y lo pasa a la función ensambladora.
3. La función ensambladora calcula el factorial de forma recursiva.
4. C++ toma el resultado y lo imprime en pantalla.

3 Código fuente

3.1 Programa principal (main.cpp)

El código en C++ hace tres cosas: declarar que la función `Mi_fact` existe en otro archivo, pedirle un número al usuario, y mostrar el resultado.

```cpp
#include <iostream>
using namespace std;

// Le decimos a C++ que esta funcion esta definida en el archivo .asm
extern "C" double Mi_fact(double x);

int main() {
    double numero;
    cout << "--- Calculadora de Factorial (ASM Recursivo) ---" << endl;
    cout << "Introduce un numero: ";
    cin >> numero;

    double resultado = Mi_fact(numero);
    cout << "El factorial de " << numero
         << " es: " << resultado << endl;
    return 0;
}
```

Listing 1: main.cpp — Entrada, salida y llamada a la función ASM

La línea **extern "C"** es el puente entre C++ y ASM. Le dice al compilador que no modifique el nombre de la función al momento de compilar, para que ambos archivos puedan encontrarse sin problemas de decoración de nombres.

3.2 Función ensambladora (factorial.asm)

Aquí está el núcleo del programa. La función recibe `x`, comprueba si es 1 o menos (caso base), y si no, se llama a sí misma con `x - 1` y multiplica el resultado por `x`.

```asm
.586
.model flat, c

.code

Mi_fact PROC

    ; Preparar el marco de pila
    PUSH    EBP
    MOV     EBP, ESP
    SUB     ESP, 8              ; Reservar 8 bytes para la variable local (x - 1.0)

    FLD     QWORD PTR [EBP+8]   ; Cargar el parametro x en la pila FPU -> ST(0) = x

    ; Comparar x con 1.0
    FLD1                         ; ST(0) = 1.0, ST(1) = x
    FCOMIP  ST(0), ST(1)         ; Comparar 1.0 con x; actualiza flags de CPU
    JAE     caso_base            ; Si 1.0 >= x (es decir x <= 1), ir al caso base

    ; x > 1: calcular x * Mi_fact(x - 1)
```

```
    FLD1                            ; ST(0) = 1.0, ST(1) = x
    FSUBP   ST(1), ST(0)            ; ST(0) = x - 1.0
    FSTP    QWORD PTR [EBP-8]       ; Guardar (x - 1.0) en variable local y vaciar FPU

    PUSH    DWORD PTR [EBP-4]       ; Pasar (x-1.0) parte alta a la pila
    PUSH    DWORD PTR [EBP-8]       ; Pasar (x-1.0) parte baja a la pila
    CALL    Mi_fact                 ; Llamada recursiva; resultado queda en ST(0)
    ADD     ESP, 8                  ; Limpiar el argumento de la pila

    FMUL    QWORD PTR [EBP+8]       ; ST(0) = Mi_fact(x-1) * x
    JMP     fin

caso_base:
    FSTP    ST(0)                   ; Descartar x de la FPU
    FLD1                            ; Retornar 1.0 en ST(0)

fin:
    MOV     ESP, EBP
    POP     EBP
    RET

Mi_fact ENDP
END
```

Listing 2: factorial.asm — Implementación recursiva con FPU

4 Cómo funciona la recursión

Cada vez que la función se llama a sí misma, guarda su estado actual en la **pila de memoria**, resuelve el subproblema más pequeño y, al regresar, retoma donde se quedó.

Para `Mi_fact(4.0)`, la cadena de llamadas es:

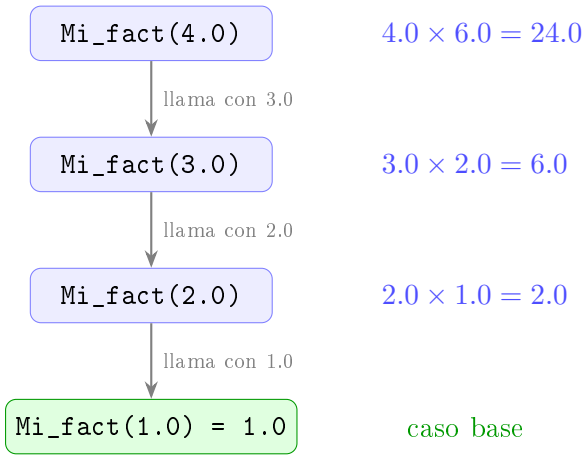

Figure 1: Cadena de llamadas recursivas para `Mi_fact(4.0) = 24`

Cada nivel espera a que el nivel inferior termine, multiplica el resultado por su `x` y lo regresa al nivel superior.

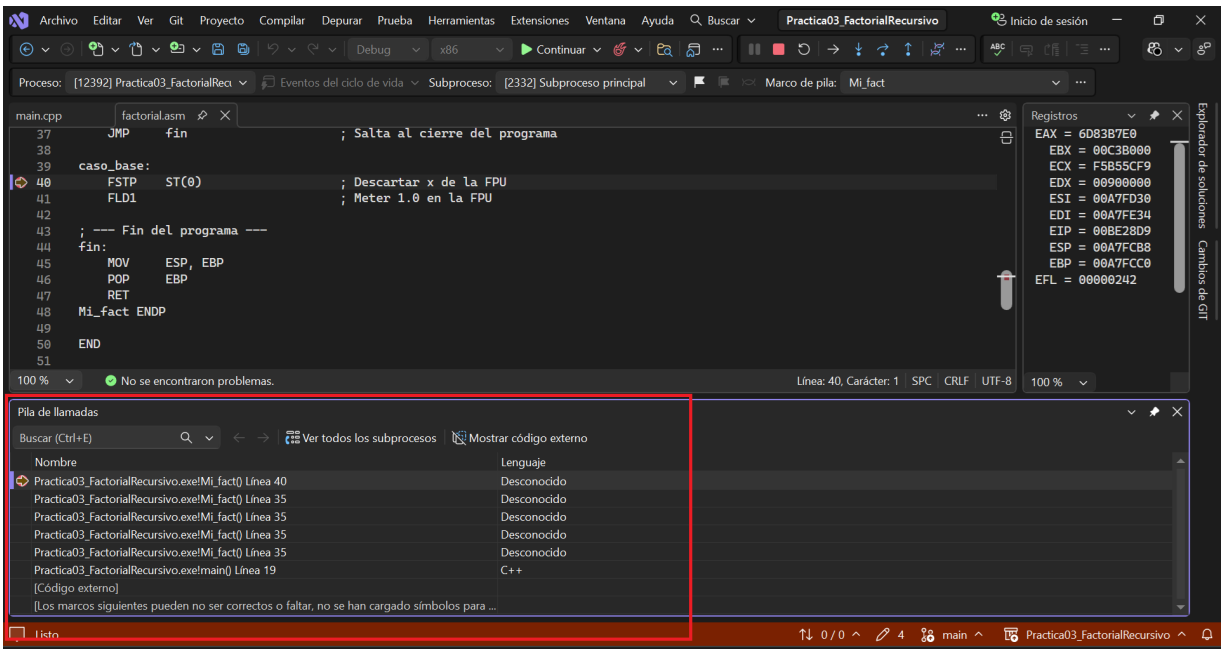

Figure 2: Ventana Call Stack de Visual Studio mostrando las llamadas recursivas apiladas de `Mi_fact`.

5 La Unidad de Punto Flotante (FPU)

El número se maneja como `double` (número con decimales de 64 bits) porque así lo requiere la interfaz con C++. Para operar con este tipo de dato, el ensamblador usa la **FPU**, un componente del procesador dedicado a la aritmética con decimales. La FPU tiene su propia pequeña pila interna de registros llamados `ST(0)` a `ST(7)`.

Las instrucciones FPU que usa esta práctica son:

Table 2: Instrucciones FPU utilizadas

Instrucción	**Qué hace en términos simples**
`FLD`	Toma un número de la memoria y lo coloca en la cima de la pila FPU (`ST(0)`).
`FLD1`	Coloca directamente el valor 1.0 en la cima de la pila FPU.
`FCOMIP`	Compara `ST(0)` con otro registro FPU, actualiza los flags de la CPU y saca `ST(0)` de la pila.
`FSUBP`	Resta los dos valores del tope de la pila FPU y saca el operando fuente.
`FSTP`	Saca el valor del tope de la pila FPU y lo guarda en memoria.
`FMUL`	Multiplica la cima de la pila FPU por un valor en memoria.

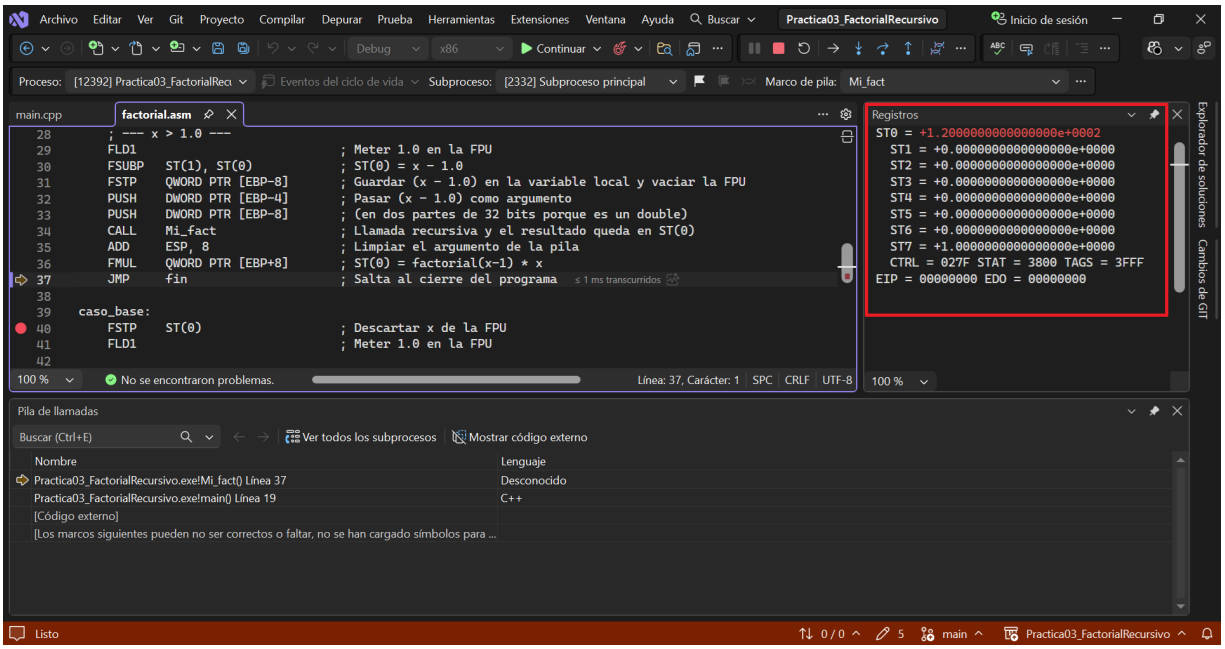

Figure 3: Ventana con los registros FPU `ST(0)`–`ST(7)`. En este caso `ST(0)` $= 1.2 \times 10^2 = 120$

6 Ejemplo de ejecución

Al correr el programa, la consola muestra:

```
--- Calculadora de Factorial (ASM Recursivo) ---
Introduce un numero: 6
El factorial de 6 es: 720
```

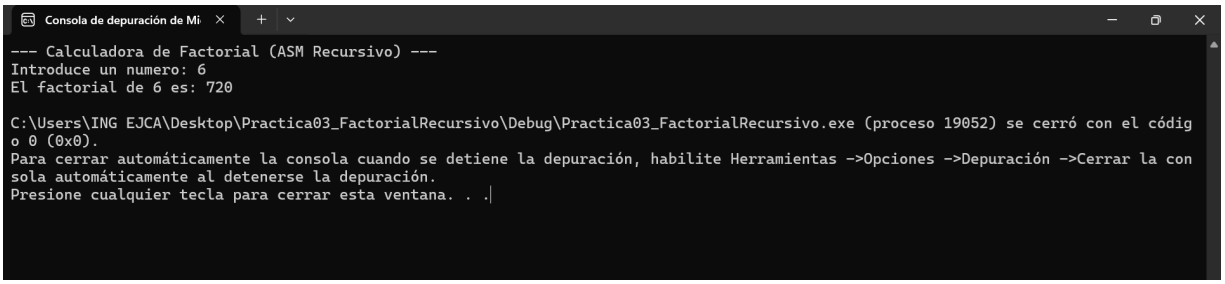

Figure 4: Consola de ejecución: el usuario introduce 6 y el programa devuelve 720.

7 Repositorio GitHub

`https://github.com/7mo-ArquitecturaComputadoras/Practica03_FactorialRecursivo`

8 Conclusiones

- Fue posible combinar C++ con ensamblador de forma sencilla gracias a `extern "C"`, que actúa como puente entre los dos lenguajes al evitar la decoración de nombres del compilador.
- La recursión en ensamblador funciona igual que en cualquier lenguaje de alto nivel: cada llamada guarda su estado en la pila, espera el resultado del nivel inferior y continúa. La diferencia es que en ensamblador todo ese manejo se hace instrucción por instrucción, de forma manual.
- El uso de la FPU permite trabajar con números de 64 bits (`double`), lo que hace a la función compatible con el tipo `double` de C++ y garantiza precisión suficiente para los factoriales de números pequeños.
- La instrucción `FCOMIP` es clave: compara dos valores en la pila FPU y actualiza directamente los flags enteros de la CPU, permitiendo usar saltos condicionales (`JAE`) sin necesidad de instrucciones adicionales de transferencia.
- Esta práctica ayuda a entender lo que realmente ocurre dentro del procesador cada vez que un programa llama a una función recursiva, algo que los lenguajes de alto nivel ocultan automáticamente.